LEARNING MAP · PYTHON BACKEND

BACKEND ROADMAP

Карта кроличьей норы

Python · PostgreSQL · FastAPI · SQLAlchemy · Alembic · pytest · Docker · Redis · Celery · Kafka

PYTHON | Sql / Postgresql | Fastapi | Sqlalchemy + Alembic | PYTEST | DOCKER | REDIS | CELERY | KAFKA

Содержание

00	BACKEND
01	PYTHON
02	SQL / POSTGRESQL
03	FASTAPI
04	SQLALCHEMY + ALEMBIC
05	PYTEST
06	DOCKER
07	REDIS
08	CELERY
09	KAFKA
10	INTEGRATION / BACKEND ARCHITECTURE
11	FULL BACKEND PROJECT
12	DEPENDENCY MAP
13	DEPTH MODEL
14	СТАТУСЫ
15	ФОРМАТ АТОМАРНОЙ ЗАДАЧИ
16	ФИНАЛЬНЫЙ CHECKLIST
17	CAPSTONE

Как пользоваться

Главный принцип:

> Тема считается изученной не тогда, когда ты её прочитал, а когда можешь объяснить её, написать код, отладить ошибку и ответить на базовые interview-вопросы.

Для каждого атомарного узла используй 6 типов работы:

- 📚 THEORY — понять концепцию
- 🎤 INTERVIEW — ответить на вопросы
- 💻 PRACTICE — решить небольшие задачи
- 🐛 DEBUGGING — найти и объяснить ошибки
- 🏗️ IMPLEMENTATION — реализовать концепцию самостоятельно
- 🚀 PROJECT — применить в реальном backend-проекте

00 BACKEND

```
BACKEND
│
├── 01. PYTHON
├── 02. SQL / POSTGRESQL
├── 03. FASTAPI
├── 04. SQLALCHEMY + ALEMBIC
├── 05. PYTEST
├── 06. DOCKER
├── 07. REDIS
├── 08. CELERY
└── 09. KAFKA
```

01 PYTHON

01.1 Fundamentals

TOPICS

- Variables
- Objects
- References
- Assignment
- Identity
- Equality
- Scope
- LEGB
- Operators
- Control Flow
- `if / elif / else`
- `for`
- `while`
- `break / continue`
- `enumerate`
- `zip`

MICROTASKS

- [ ] Понять, что такое переменная в Python
- [ ] Понять object/reference model
- [ ] Разобраться с `==` и `is`
- [ ] Разобраться с truthy/falsy
- [ ] Понять `and / or / not`
- [ ] Написать условия
- [ ] Написать циклы
- [ ] Использовать `enumerate`
- [ ] Использовать `zip`

INTERVIEW

- [ ] Что такое переменная в Python?
- [ ] Что означает `a = b = []`?
- [ ] Что такое reference?
- [ ] В чём разница `==` и `is`?
- [ ] Что такое mutable object?
- [ ] Что такое LEGB?
- [ ] Что такое truthy/falsy?
- [ ] Что возвращает функция без `return`?

PRACTICE

- [ ] FizzBuzz
- [ ] Поиск максимума
- [ ] Подсчёт элементов
- [ ] Palindrome
- [ ] Frequency counter
- [ ] Обработка вложенных структур

01.2 Data Types & Collections

TYPES

- [ ] `int`
- [ ] `float`
- [ ] `bool`
- [ ] `complex`
- [ ] `None`
- [ ] `str`
- [ ] `list`
- [ ] `tuple`
- [ ] `set`
- [ ] `dict`

ДЛЯ КАЖДОГО ТИПА

- [ ] Создание
- [ ] Чтение
- [ ] Изменение
- [ ] Удаление
- [ ] Mutability
- [ ] Indexing
- [ ] Slicing
- [ ] Iteration
- [ ] Complexity
- [ ] Memory
- [ ] Hashability
- [ ] Common pitfalls

INTERVIEW

- [ ] `list` vs `tuple`
- [ ] `list` vs `set`
- [ ] `dict` vs `set`
- [ ] Почему dict key должен быть hashable?
- [ ] Что значит immutable?
- [ ] Почему tuple может быть ключом dict?
- [ ] Может ли tuple содержать mutable object?
- [ ] Что произойдёт с `[[1]] * 3`?
- [ ] Какова сложность `x in list`?
- [ ] Какова сложность lookup в dict/set?

PRACTICE

- [ ] Удалить дубликаты
- [ ] Frequency counter
- [ ] Группировка
- [ ] Flatten nested list
- [ ] Реализовать stack
- [ ] Реализовать queue
- [ ] Реализовать простой cache

01.3 Functions

TOPICS

- [ ] `def`
- [ ] `return`
- [ ] Parameters / arguments
- [ ] Default arguments
- [ ] `*args`
- [ ] `**kwargs`
- [ ] Keyword-only arguments
- [ ] Positional-only arguments
- [ ] Lambda
- [ ] Scope
- [ ] Closures
- [ ] First-class functions

INTERVIEW

- [ ] Что такое first-class function?
- [ ] Что такое closure?
- [ ] Что такое `*args` и `**kwargs`?
- [ ] Positional vs keyword arguments?
- [ ] Что такое mutable default argument?
- [ ] Почему `def f(x=[]): ...` опасен?

PRACTICE

- [ ] Closure-based counter
- [ ] Memoization
- [ ] Retry decorator
- [ ] Timing decorator

01.4 Exceptions

TOPICS

- [ ] `Exception`
- [ ] `BaseException`
- [ ] `try`
- [ ] `except`
- [ ] `else`
- [ ] `finally`
- [ ] `raise`
- [ ] Custom exceptions
- [ ] Exception chaining

INTERVIEW

- [ ] Зачем нужен `finally`?
- [ ] Когда использовать `raise`?
- [ ] `Exception` vs `BaseException`?

- [ ] Что такое exception chaining?
- [ ] Что будет, если ошибка возникнет внутри `finally`?

PRACTICE

- [ ] Собственные exceptions
- [ ] Retry logic
- [ ] Validation layer
- [ ] Error handling

01.5 OOP

TOPICS

- [ ] Class
- [ ] Object
- [ ] Instance
- [ ] Attribute
- [ ] Method
- [ ] `classmethod`
- [ ] `staticmethod`
- [ ] `property`
- [ ] Inheritance
- [ ] Composition
- [ ] Polymorphism
- [ ] Encapsulation
- [ ] Abstraction
- [ ] MRO
- [ ] `super()`

MAGIC METHODS

- [ ] `__init__`
- [ ] `__new__`
- [ ] `__str__`
- [ ] `__repr__`
- [ ] `__eq__`
- [ ] `__hash__`
- [ ] `__len__`
- [ ] `__iter__`
- [ ] `__next__`
- [ ] `__enter__`
- [ ] `__exit__`

INTERVIEW

- [ ] Что такое MRO?
- [ ] Как работает `super()`?
- [ ] Inheritance vs composition?
- [ ] `classmethod` vs `staticmethod`?
- [ ] `__new__` vs `__init__`?
- [ ] Почему `__eq__` связан с `__hash__`?
- [ ] Что такое duck typing?

01.6 Iterators / Generators

TOPICS

- [ ] Iterable
- [ ] Iterator
- [ ] `iter()`
- [ ] `next()`
- [ ] `yield`
- [ ] Generator
- [ ] Generator expression
- [ ] Lazy evaluation

INTERVIEW

- [ ] Iterable vs iterator?
- [ ] Что делает `yield`?
- [ ] Почему generator экономит память?
- [ ] Что происходит при `next()`?
- [ ] Что такое `StopIteration`?

PRACTICE

- [ ] Написать свой iterator
- [ ] Generator для чтения файла
- [ ] Streaming parser
- [ ] Lazy pipeline

01.7 Decorators

TOPICS

- [ ] Function decorator
- [ ] Class decorator
- [ ] `@` syntax
- [ ] `functools.wraps`
- [ ] Parameterized decorator

PRACTICE

- [ ] `@timer`
- [ ] `@retry`
- [ ] `@auth_required`
- [ ] `@cache`

01.8 Context Managers

TOPICS

- [ ] `with`
- [ ] `__enter__`
- [ ] `__exit__`
- [ ] `contextlib`
- [ ] `contextmanager`

PRACTICE

- [ ] File context
- [ ] Transaction context
- [ ] Lock context

01.9 Typing

TOPICS

- [ ] Type hints
- [ ] `Optional`
- [ ] `Union`
- [ ] `Literal`
- [ ] `Any`
- [ ] `Generic`
- [ ] `TypeVar`
- [ ] `Protocol`
- [ ] `TypedDict`
- [ ] `Callable`

INTERVIEW

- [ ] Зачем нужен typing?
- [ ] `Any` vs `object`?
- [ ] `Protocol` vs ABC?
- [ ] Что такое structural typing?

01.10 Modules / Packages

- [ ] `import`
- [ ] `from ... import`
- [ ] `__init__.py`
- [ ] `__name__`
- [ ] `__main__`
- [ ] Module
- [ ] Package
- [ ] Circular imports

01.11 Memory / Internals

TOPICS

- [ ] Python object model
- [ ] Garbage collector
- [ ] CPython
- [ ] References
- [ ] Cyclic references
- [ ] GIL
- [ ] Reference counting
- [ ] Memory allocation

INTERVIEW

- [ ] Как Python управляет памятью?
- [ ] Что такое cyclic reference?
- [ ] Что такое reference counting?
- [ ] Что такое GIL?
- [ ] Зачем нужен GC?

01.12 Concurrency

- [ ] Process
- [ ] Concurrency
- [ ] IO-bound
- [ ] Thread
- [ ] Parallelism
- [ ] `threading`
- [ ] Coroutine
- [ ] CPU-bound
- [ ] `multiprocessing`

KEY QUESTION

- [ ] Когда использовать thread, process или async?

01.13 Asyncio

TOPICS

- [ ] Event loop
- [ ] Task
- [ ] Blocking code
- [ ] Coroutine
- [ ] Future
- [ ] Non-blocking I/O
- [ ] `async`
- [ ] `gather`
- [ ] Cancellation
- [ ] `await`
- [ ] `create_task`
- [ ] Timeout

PRACTICE

- [ ] Async HTTP requests
- [ ] Timeout handling
- [ ] Concurrent requests
- [ ] Cancellation

01.14 Testing

- [ ] Unit test
- [ ] Mock
- [ ] Assertion
- [ ] Integration test
- [ ] Fixture
- [ ] Test isolation

01.15 Packaging / Tooling

- [ ] `venv`
- [ ] Dependencies
- [ ] Linting
- [ ] `pip`
- [ ] Lock files
- [ ] Project structure
- [ ] `pyproject.toml`
- [ ] Formatting

02 SQL / POSTGRESQL

02.1 Relational Databases

- [ ] Database
- [ ] Table
- [ ] Row
- [ ] Column
- [ ] Primary key
- [ ] Foreign key
- [ ] Relationship
- [ ] Schema

02.2 SQL Fundamentals

- [ ] `SELECT`
- [ ] `WHERE`
- [ ] `ORDER BY`
- [ ] `LIMIT`
- [ ] `OFFSET`
- [ ] `DISTINCT`
- [ ] `INSERT`
- [ ] `UPDATE`
- [ ] `DELETE`

PRACTICE

Создать:

- [ ] users
- [ ] orders
- [ ] products
- [ ] order_items

И написать минимум 50 SQL-запросов.

02.3 JOINs

- [ ] `INNER JOIN`
- [ ] `LEFT JOIN`
- [ ] `RIGHT JOIN`
- [ ] `FULL JOIN`
- [ ] `CROSS JOIN`
- [ ] `SELF JOIN`

INTERVIEW

- [ ] `INNER JOIN` vs `LEFT JOIN`
- [ ] Что происходит, если справа нет записи?
- [ ] Почему `LEFT JOIN + WHERE` иногда превращается в `INNER JOIN`?
- [ ] Когда нужен self join?

02.4 Aggregations

- [ ] `COUNT`
- [ ] `SUM`
- [ ] `AVG`
- [ ] `MIN`
- [ ] `MAX`
- [ ] `GROUP BY`
- [ ] `HAVING`

PRACTICE

- [ ] Найти пользователей с >5 заказами
- [ ] Посчитать продажи по продуктам
- [ ] Посчитать средний чек
- [ ] Найти top-N клиентов

02.5 Subqueries / CTE

- [ ] Subquery
- [ ] `WITH`
- [ ] `EXISTS`
- [ ] Correlated subquery
- [ ] Recursive CTE
- [ ] `IN`

INTERVIEW

- [ ] `EXISTS` vs `IN`
- [ ] CTE vs subquery
- [ ] Когда CTE улучшает читаемость?

02.6 Database Design

- [ ] Normalization
- [ ] 3NF
- [ ] One-to-many
- [ ] 1NF
- [ ] Denormalization
- [ ] Many-to-many
- [ ] 2NF
- [ ] One-to-one
- [ ] Junction table

PRACTICE

Спроектировать:

```
User
  ↓
Order
  ↓
OrderItem
  ↓
Product
```

02.7 Constraints

- [ ] `PRIMARY KEY`
- [ ] `UNIQUE`
- [ ] `CHECK`
- [ ] `FOREIGN KEY`
- [ ] `NOT NULL`
- [ ] `DEFAULT`

02.8 Indexes

TOPICS

- [ ] Index
- [ ] Selectivity
- [ ] Query planner
- [ ] B-tree
- [ ] Index scan
- [ ] Covering index
- [ ] Composite index
- [ ] Sequential scan

INTERVIEW

- [ ] Зачем нужен index?
- [ ] Почему index ускоряет SELECT?
- [ ] Почему index замедляет INSERT?
- [ ] Почему нельзя индексировать всё?
- [ ] Как выбрать порядок колонок composite index?
- [ ] Почему PostgreSQL может не использовать index?

PRACTICE

- [ ] Создать index
- [ ] Сравнить query с index
- [ ] Сравнить query без index
- [ ] Использовать `EXPLAIN ANALYZE`

02.9 Transactions

- [ ] `BEGIN`
- [ ] `SAVEPOINT`
- [ ] `COMMIT`
- [ ] ACID
- [ ] `ROLLBACK`

02.10 Isolation / MVCC

- [ ] Dirty read
- [ ] READ COMMITTED
- [ ] MVCC
- [ ] Non-repeatable read
- [ ] REPEATABLE READ
- [ ] Snapshot
- [ ] Phantom read
- [ ] SERIALIZABLE
- [ ] Visibility

INTERVIEW

- [ ] Что такое ACID?
- [ ] Что такое phantom read?
- [ ] Что такое transaction?
- [ ] Как PostgreSQL реализует concurrency?
- [ ] Что такое isolation level?
- [ ] Что такое MVCC?
- [ ] Что такое dirty read?

02.11 Locks

- [ ] Row lock
- [ ] Deadlock
- [ ] Table lock
- [ ] `SELECT FOR UPDATE`

PRACTICE

- [ ] Смоделировать два concurrent transaction
- [ ] Получить race condition
- [ ] Исправить через lock

02.12 Query Optimization

- [ ] `EXPLAIN`
- [ ] Index Scan
- [ ] Merge Join
- [ ] `EXPLAIN ANALYZE`
- [ ] Nested Loop
- [ ] Cost
- [ ] Seq Scan
- [ ] Hash Join
- [ ] Cardinality

PRACTICE

- [ ] Найти медленный query
- [ ] Создать index
- [ ] Посмотреть execution plan
- [ ] Сравнить latency

02.13 PostgreSQL-specific

- [ ] JSONB
- [ ] Arrays
- [ ] UUID
- [ ] Timestamps
- [ ] Full-text search
- [ ] Extensions
- [ ] Schemas
- [ ] Views
- [ ] Materialized views

03 FASTAPI

03.1 HTTP

- [ ] Request
- [ ] Response
- [ ] Method
- [ ] URL
- [ ] Headers
- [ ] Body
- [ ] Status code
- [ ] Cookies
- [ ] Query parameters
- [ ] Path parameters

METHODS

- [ ] GET
- [ ] POST
- [ ] PUT
- [ ] PATCH
- [ ] DELETE
- [ ] OPTIONS
- [ ] HEAD

INTERVIEW

- [ ] PUT vs PATCH?
- [ ] GET vs POST?
- [ ] 401 vs 403?
- [ ] 400 vs 422?
- [ ] 404 vs 204?
- [ ] Что такое idempotency?

03.2 Routing

- [ ] `APIRouter`
- [ ] Path params
- [ ] Query params
- [ ] Request body
- [ ] `response_model`
- [ ] `status_code`

PRACTICE

- [ ] `/users`
- [ ] `/users/{id}`
- [ ] `/orders`
- [ ] `/orders/{id}`

03.3 Pydantic

- [ ] `BaseModel`
- [ ] Validation
- [ ] Serialization
- [ ] Nested models
- [ ] Field validators
- [ ] Model validators
- [ ] Optional fields
- [ ] Defaults

03.4 Dependency Injection

- [ ] `Depends`
- [ ] Dependency
- [ ] Nested dependency
- [ ] Database dependency
- [ ] Auth dependency
- [ ] Dependency scopes

PRACTICE

- [ ] `get_db()`
- [ ] `get_current_user()`
- [ ] `require_admin()`

03.5 Error Handling

- [ ] `HTTPException`
- [ ] Custom exceptions
- [ ] Exception handlers
- [ ] Validation errors
- [ ] Global handlers

03.6 Middleware

- [ ] Middleware
- [ ] Request lifecycle
- [ ] Response lifecycle
- [ ] Logging
- [ ] Timing
- [ ] CORS

03.7 Authentication

- [ ] Password hashing
- [ ] JWT
- [ ] Access token
- [ ] Refresh token
- [ ] OAuth2
- [ ] Expiration
- [ ] Revocation

INTERVIEW

- [ ] JWT vs session?
- [ ] Что хранится в JWT?
- [ ] Почему нельзя хранить password plaintext?
- [ ] Access vs refresh token?
- [ ] Как отозвать JWT?

03.8 Authorization

- [ ] Roles
- [ ] Permissions
- [ ] RBAC
- [ ] Resource ownership

03.9 Async API

```
FastAPI
 ↓
async def
 ↓
asyncio
 ↓
event loop
 ↓
async SQLAlchemy
```

- [ ] Понять каждую ступень
- [ ] Найти blocking code
- [ ] Объяснить последствия blocking I/O

03.10 Production API

- [ ] Logging
- [ ] Configuration
- [ ] Environment variables
- [ ] Health checks
- [ ] Graceful shutdown
- [ ] CORS
- [ ] Rate limiting
- [ ] OpenAPI

04 SQLALCHEMY + ALEMBIC

04.1 SQLAlchemy Core

- [ ] Engine
- [ ] Connection
- [ ] SQL expressions
- [ ] `select()`
- [ ] `insert()`
- [ ] `update()`
- [ ] `delete()`

04.2 ORM

- [ ] DeclarativeBase
- [ ] `Mapped`
- [ ] `mapped_column`
- [ ] `relationship`
- [ ] Session

04.3 Models

- [ ] User
- [ ] Order
- [ ] Product
- [ ] OrderItem
- [ ] Constraints
- [ ] Relationships

04.4 Relationships

- [ ] One-to-one
- [ ] One-to-many
- [ ] Many-to-many
- [ ] `back_populates`
- [ ] Cascade

04.5 Session

- [ ] Session lifecycle
- [ ] Identity map
- [ ] `flush`
- [ ] `commit`
- [ ] `rollback`
- [ ] `refresh`
- [ ] `expire`

INTERVIEW

- [ ] `flush` vs `commit`
- [ ] `commit` vs `refresh`
- [ ] Что такое identity map?
- [ ] Когда объект попадает в DB?
- [ ] Что происходит при rollback?

04.6 Loading Strategies

- [ ] Lazy loading
- [ ] `joinedload`
- [ ] `selectinload`
- [ ] `contains_eager`
- [ ] N+1 problem

PRACTICE

- [ ] Найти N+1

- [ ] Посчитать SQL queries
- [ ] Исправить eager loading

04.7 Transactions

- [ ] Session transaction
- [ ] `begin`
- [ ] `commit`
- [ ] `rollback`
- [ ] Nested transaction

04.8 Async SQLAlchemy

- [ ] AsyncEngine
- [ ] AsyncSession
- [ ] Async queries
- [ ] Connection pool
- [ ] Async transaction

04.9 Alembic

- [ ] Migration
- [ ] Revision
- [ ] Upgrade
- [ ] Downgrade
- [ ] Autogenerate
- [ ] Migration history
- [ ] Migration conflicts

PRACTICE

```
User
 ↓
migration
 ↓
add email
 ↓
migration
 ↓
change constraint
 ↓
rollback
```

05 PYTEST

05.1 Fundamentals

- [ ] Test structure
- [ ] Test discovery
- [ ] `assert`
- [ ] Markers

05.2 Fixtures

- [ ] Fixture
- [ ] Dependency
- [ ] Scope
- [ ] `autouse`
- [ ] Yield fixture

05.3 Parametrization

- [ ] `pytest.mark.parametrize`
- [ ] Test matrix

05.4 Mocking

- [ ] Mock
- [ ] patch
- [ ] MagicMock
- [ ] monkeypatch

05.5 Async

- [ ] Async test
- [ ] AsyncMock
- [ ] Async fixtures

05.6 Unit Tests

- [ ] Function
- [ ] Service
- [ ] Business logic

05.7 Integration Tests

- [ ] API
- [ ] Redis
- [ ] PostgreSQL
- [ ] Celery

05.8 Full API/DB testing

```
HTTP request
 ↓
FastAPI
 ↓
Service
 ↓
SQLAlchemy
 ↓
PostgreSQL
```

06 DOCKER

06.1 Core

- [ ] Image
- [ ] Container
- [ ] Registry
- [ ] Layer
- [ ] Dockerfile

06.2 Dockerfile

- [ ] `FROM`
- [ ] `RUN`
- [ ] `COPY`
- [ ] `WORKDIR`
- [ ] `ENV`
- [ ] `EXPOSE`
- [ ] `CMD`
- [ ] `ENTRYPOINT`

06.3 Volumes

- [ ] Named volume
- [ ] Bind mount
- [ ] Persistent data

06.4 Networks

- [ ] Bridge
- [ ] Container DNS
- [ ] Service discovery
- [ ] Ports

06.5 Docker Compose

Собрать:

- [ ] FastAPI
- [ ] PostgreSQL
- [ ] Redis
- [ ] Celery worker
- [ ] Kafka

06.6 Production concepts

- [ ] Multi-stage builds
- [ ] Health checks
- [ ] Resource limits
- [ ] Secrets/configuration
- [ ] Container lifecycle

07 REDIS

07.1 Data Structures

- [ ] String
- [ ] List
- [ ] Set
- [ ] Hash
- [ ] Sorted Set

07.2 TTL

- [ ] `EXPIRE`
- [ ] `TTL`
- [ ] Expiration

07.3 Caching

- [ ] Cache-aside
- [ ] Cache hit
- [ ] Cache miss
- [ ] Cache invalidation
- [ ] TTL

INTERVIEW

- [ ] Почему cache invalidation сложна?
- [ ] Что такое cache stampede?
- [ ] Когда Redis не стоит использовать как cache?

07.4 Atomic Operations

- [ ] `INCR`
- [ ] `SETNX`
- [ ] Transactions
- [ ] `WATCH`

07.5 Distributed Locks

- [ ] Lock
- [ ] TTL
- [ ] Race condition
- [ ] Distributed lock

07.6 Pub/Sub

- [ ] Publisher
- [ ] Subscriber
- [ ] Channel
- [ ] Message

07.7 Redis + Python

PRACTICE

```
FastAPI
 ↓
Redis
 ↓
Cache
```

08 CELERY

08.1 Architecture

```
Producer
   ↓
Broker
   ↓
Worker
   ↓
Result Backend
```

- [ ] Producer
- [ ] Broker
- [ ] Worker
- [ ] Consumer
- [ ] Result backend

08.2 Tasks

- [ ] Task
- [ ] `delay()`
- [ ] `apply_async()`
- [ ] Task arguments
- [ ] Task result

08.3 Workers

- [ ] Worker
- [ ] Concurrency
- [ ] Prefetch
- [ ] Acknowledgement

08.4 Broker

- [ ] Redis as broker
- [ ] Broker vs result backend

08.5 Retries

- [ ] Retry
- [ ] Exponential backoff
- [ ] Max retries
- [ ] Failure handling

08.6 Scheduling

- [ ] Periodic tasks
- [ ] Celery Beat

08.7 Reliability

- [ ] At-least-once execution
- [ ] Duplicate execution
- [ ] Idempotency
- [ ] Task failure
- [ ] ACK
- [ ] Retry

INTERVIEW

- [ ] Что произойдёт, если worker умер после выполнения задачи, но до acknowledgement?

- [ ] Почему task должна быть idempotent?
- [ ] Что произойдёт при повторном выполнении task?

08.8 Project

```
POST /reports
       ↓
FastAPI
       ↓
Celery task
       ↓
Redis
       ↓
Worker
       ↓
Generate report
       ↓
PostgreSQL
```

09 KAFKA

09.1 Messaging Concepts

- [ ] Producer
- [ ] Broker
- [ ] Consumer
- [ ] Topic
- [ ] Message

09.2 Topics

- [ ] Topic
- [ ] Partition
- [ ] Replica

09.3 Partitions

- [ ] Partition key
- [ ] Parallelism
- [ ] Ordering
- [ ] Scaling

INTERVIEW

- [ ] Зачем нужны partitions?
- [ ] Почему порядок гарантируется внутри partition?
- [ ] Как key влияет на partition?

09.4 Producer

- [ ] Send
- [ ] Retries
- [ ] Batching
- [ ] Partitioning
- [ ] `acks`

09.5 Consumer

- [ ] Poll
- [ ] Offset
- [ ] Message
- [ ] Commit

09.6 Consumer Groups

- [ ] Consumer group
- [ ] Partition assignment
- [ ] Consumer
- [ ] Rebalance

INTERVIEW

- [ ] Почему consumer'ы одной group читают разные partitions?
- [ ] Что происходит при добавлении consumer?
- [ ] Что происходит при удалении consumer?

09.7 Offsets

- [ ] Offset
- [ ] Commit

- [ ] Auto commit
- [ ] Manual commit
- [ ] Reprocessing

09.8 Replication

- [ ] Leader
- [ ] Follower
- [ ] Replication factor
- [ ] ISR

09.9 Delivery Semantics

- [ ] At-most-once
- [ ] At-least-once
- [ ] Exactly-once

CRITICAL CONCEPT

```
delivery guarantee
        ≠
business-level exactly-once
```

09.10 Ordering

- [ ] Global ordering
- [ ] Partition ordering
- [ ] Key-based ordering

09.11 Reliability

- [ ] Producer retries
- [ ] Consumer retries
- [ ] Dead letter topic
- [ ] Idempotent consumer
- [ ] Duplicate messages

09.12 Kafka + Python

```
FastAPI
   ↓
Kafka Producer
   ↓
orders topic
   ↓
Kafka
   ↓
Consumer
   ↓
PostgreSQL
```

10 INTEGRATION / BACKEND ARCHITECTURE

Этот раздел нужен после изучения отдельных технологий.

10.1 REST API + Database

```
Client
  ↓
FastAPI
  ↓
Dependency Injection
  ↓
Service
  ↓
SQLAlchemy
  ↓
PostgreSQL
```

PROJECT 01 — CRUD API

- [ ] Users
- [ ] Authentication
- [ ] Orders
- [ ] PostgreSQL
- [ ] SQLAlchemy
- [ ] Alembic
- [ ] pytest
- [ ] Docker

10.2 Cache

```
Client
  ↓
FastAPI
  ↓
Redis
  ↓
PostgreSQL
```

PROJECT 02 — CACHED API

- [ ] Cache-aside
- [ ] TTL
- [ ] Cache invalidation
- [ ] Cache stampede analysis
- [ ] Tests

10.3 Background Jobs

```
FastAPI
  ↓
Celery
  ↓
Redis
  ↓
Worker
  ↓
PostgreSQL
```

PROJECT 03 — ASYNC PROCESSING

- [ ] Background task
- [ ] Retry
- [ ] Idempotency
- [ ] Failure handling
- [ ] Task monitoring

10.4 Event-driven architecture

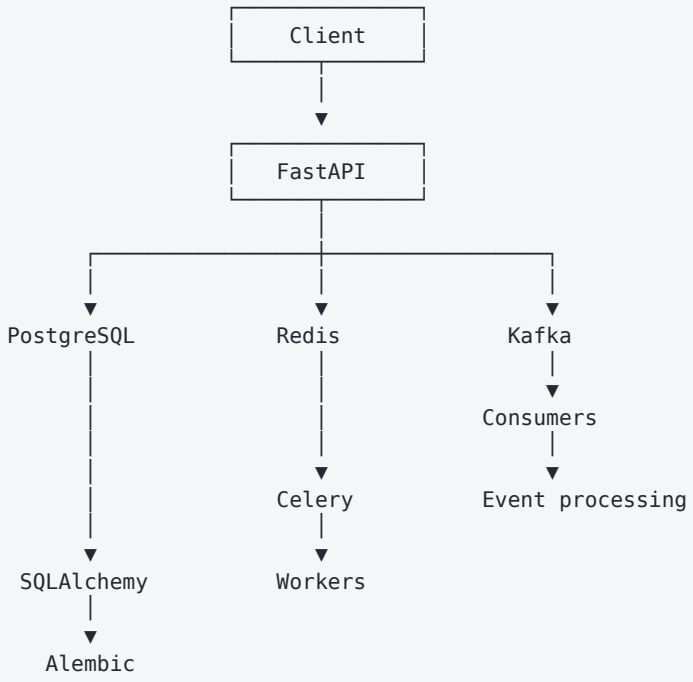
```
FastAPI
   ↓
Kafka Producer
   ↓
Kafka Topic
   ↓
Consumer Group
   ↓
PostgreSQL
```

PROJECT 04 — EVENT-DRIVEN BACKEND

- [ ] Order created event
- [ ] Kafka producer
- [ ] Kafka consumer
- [ ] Partitions
- [ ] Consumer groups
- [ ] Offset management
- [ ] Retry
- [ ] Idempotency
- [ ] Dead letter topic

11 FULL BACKEND PROJECT

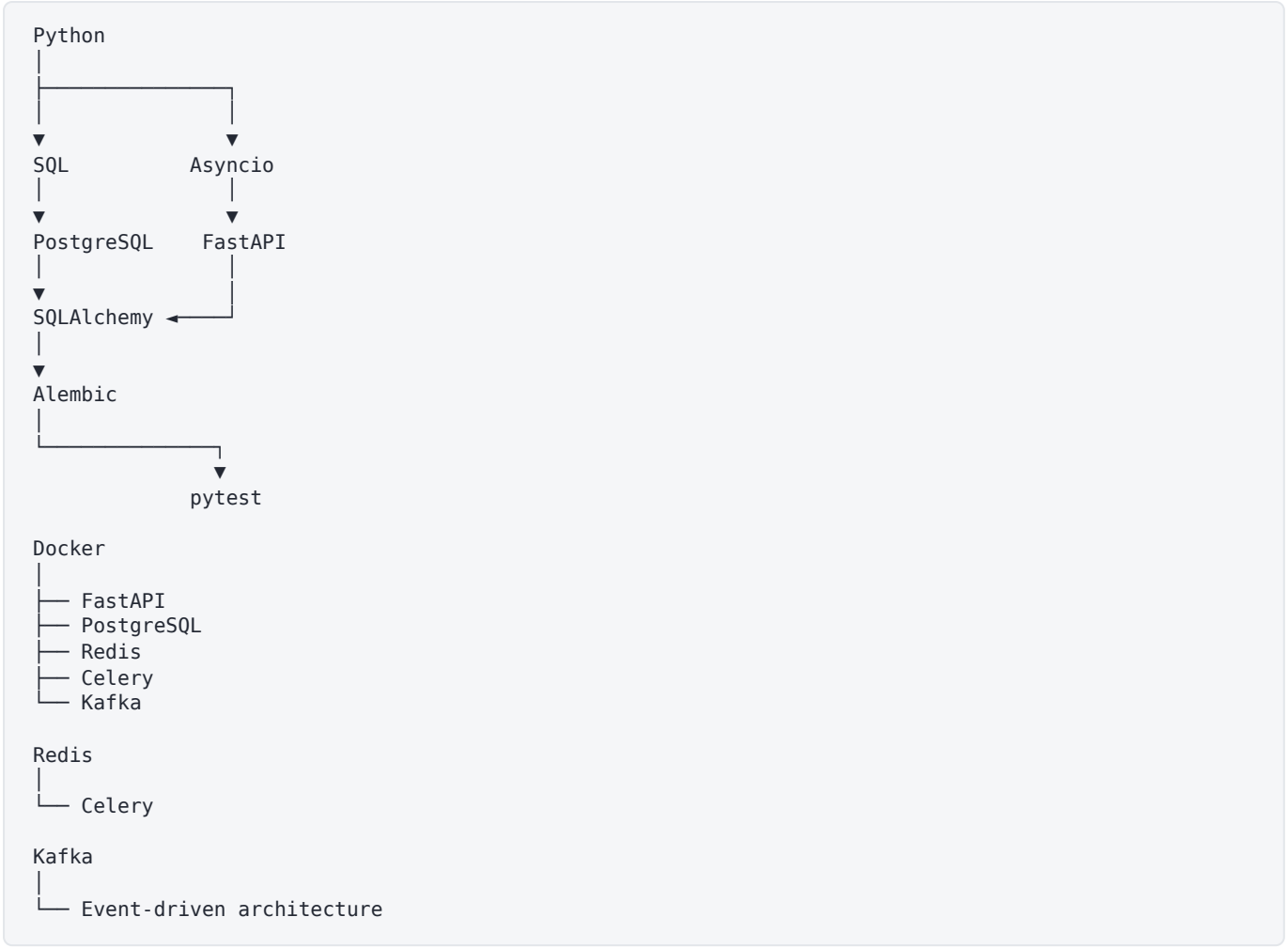
Project 05 — Production-like Backend



REQUIREMENTS

- [ ] FastAPI
- [ ] PostgreSQL
- [ ] SQLAlchemy
- [ ] Alembic
- [ ] Pydantic
- [ ] pytest
- [ ] Docker
- [ ] Redis
- [ ] Celery
- [ ] Kafka
- [ ] Authentication
- [ ] Authorization
- [ ] Logging
- [ ] Error handling
- [ ] Health checks
- [ ] Caching
- [ ] Background jobs
- [ ] Events
- [ ] Retries
- [ ] Idempotency
- [ ] Integration tests

12 DEPENDENCY MAP



13 DEPTH MODEL

Для каждого Topic используй уровень глубины:

```
D0 — не знаю
D1 — слышал термин
D2 — могу объяснить простыми словами
D3 — могу написать пример
D4 — могу применить в backend
D5 — могу отладить проблему
D6 — могу объяснить внутреннее устройство
D7 — могу объяснить trade-offs и архитектурные последствия
```

Пример:

```
PostgreSQL / Indexes

D0  — не знаю
D1  — знаю, что index существует
D2  — могу объяснить зачем нужен
D3  — могу создать index
D4  — умею использовать в проекте
D5  — умею найти проблему через EXPLAIN
D6  — понимаю B-tree / planner
D7  — понимаю trade-offs и архитектурные последствия
```

14 СТАТУСЫ

Рекомендуемые статусы:

```
NOT_STARTED
LEARNING
PRACTICING
REVIEW
MASTERED
BLOCKED
```

Не ставь `MASTERED`, пока не выполнены:

- [ ] могу объяснить
- [ ] могу ответить на вопросы
- [ ] могу написать
- [ ] могу отладить
- [ ] могу применить в проекте

15 ФОРМАТ АТОМАРНОЙ ЗАДАЧИ

Для каждой темы можно создавать карточку:

```
Topic:
Python → Data Types → Dict → Hashability

THEORY
□ Что такое hash?
□ Что такое hashable object?
□ Почему list unhashable?
□ Почему tuple иногда hashable?
□ Как hash связан с dict?

INTERVIEW
□ Почему ключ dict должен быть hashable?
□ Можно ли использовать tuple как key?
□ Можно ли использовать custom object как key?
□ Что произойдёт при изменении hashable объекта?

PRACTICE
□ Создать dict с разными типами ключей
□ Создать custom hashable object

DEBUGGING
□ Найти и объяснить TypeError: unhashable type

IMPLEMENTATION
□ Реализовать простой hash-table concept

PROJECT
□ Использовать dict как cache

CONFIDENCE
Theory:      0/5
Interview:   0/5
Practice:    0/5
Debugging:   0/5
Internals:   0/5

DEPTH
D0
```

16 ФИНАЛЬНЫЙ CHECKLIST

Python

- [ ] Fundamentals
- [ ] Data Types
- [ ] Collections
- [ ] Functions
- [ ] Exceptions
- [ ] OOP
- [ ] Iterators
- [ ] Generators
- [ ] Decorators
- [ ] Context Managers
- [ ] Typing
- [ ] Modules
- [ ] Memory
- [ ] Concurrency
- [ ] Asyncio
- [ ] Testing
- [ ] Packaging

PostgreSQL

- [ ] SQL
- [ ] JOIN
- [ ] Aggregation
- [ ] Subqueries
- [ ] CTE
- [ ] Database Design
- [ ] Constraints
- [ ] Indexes
- [ ] Transactions
- [ ] Isolation
- [ ] MVCC
- [ ] Locks
- [ ] Query Optimization
- [ ] PostgreSQL features

FastAPI

- [ ] HTTP
- [ ] Routing
- [ ] Request/Response
- [ ] Pydantic
- [ ] Dependency Injection
- [ ] Error Handling
- [ ] Middleware
- [ ] Authentication
- [ ] Authorization
- [ ] Async
- [ ] Production

SQLAlchemy / Alembic

- [ ] Core
- [ ] ORM
- [ ] Models
- [ ] Relationships
- [ ] Session
- [ ] Loading
- [ ] N+1
- [ ] Transactions
- [ ] Async
- [ ] Migrations

pytest

- [ ] Fundamentals
- [ ] Fixtures
- [ ] Parametrization
- [ ] Mocking
- [ ] Async
- [ ] Unit
- [ ] Integration
- [ ] API/DB

Docker

- [ ] Images
- [ ] Containers
- [ ] Dockerfile
- [ ] Layers
- [ ] Volumes
- [ ] Networks
- [ ] Compose
- [ ] Production

Redis

- [ ] Data structures
- [ ] TTL
- [ ] Cache

- [ ] Atomicity
- [ ] Locks
- [ ] Pub/Sub
- [ ] Python integration

Celery

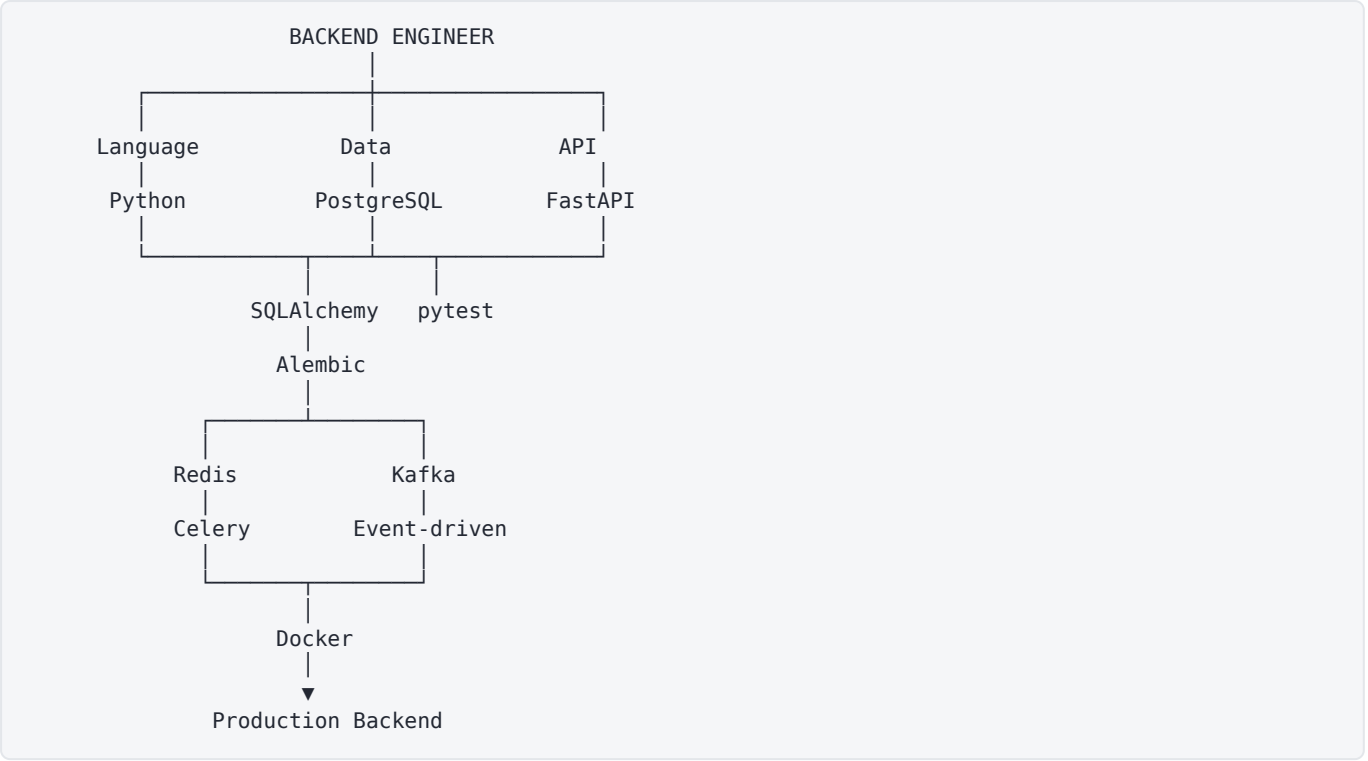
- [ ] Architecture
- [ ] Tasks
- [ ] Workers
- [ ] Broker
- [ ] Result backend
- [ ] Retries
- [ ] Scheduling
- [ ] Reliability
- [ ] Idempotency

Kafka

- [ ] Messaging
- [ ] Topics
- [ ] Partitions
- [ ] Producer
- [ ] Consumer
- [ ] Consumer groups
- [ ] Offsets
- [ ] Replication
- [ ] Delivery semantics
- [ ] Ordering
- [ ] Reliability
- [ ] Python integration

17 CAPSTONE

Финальная цель:



Главная задача — не просто закрыть все чекбоксы, а постепенно увеличивать `Depth` каждого узла и регулярно связывать отдельные технологии через проекты.